

# hiero-hcs-replay

*Realistic Load Testing for Hedera Applications*

Hiero Developer Tools Track | Hackathon Submission | March 2026

**Taras Smereka**

[github.com/kaldun-tech/hiero-hcs-replay](https://github.com/kaldun-tech/hiero-hcs-replay)

# The Problem

*Synthetic load testing produces unrealistic results*

## Real HCS traffic is bursty and irregular

---

- **Uniform load generators miss production edge cases**
- Real HCS topics: message bursts followed by silence
- Production failures often occur during traffic spikes
- Developers need to reproduce real-world conditions to catch bugs

## The Gap

---

| Current Approach             | The Problem                                         |
|------------------------------|-----------------------------------------------------|
| Synthetic traffic generators | Uniform intervals — not how real Hedera apps behave |
| Manual timing scripts        | One-off, not reusable, no statistical analysis      |
| No replay tooling exists     | Developers can't reproduce real production patterns |

# The Solution

*Capture real timing patterns. Replay at any speed.*

## Simple Pipeline

---

```
HCS Topic → Mirror Node API → FetchTiming() → TimingData → Replay()
```

## What it does

---

- Fetch real timing data from any public HCS topic on Mainnet, Testnet, or Previewnet
- Statistical analysis: min, max, avg, P50, P90, P99 latency
- Two replay modes: Sequential (exact order) or Sample (statistical distribution)
- Configurable speedup: 1x, 10x, 100x — run hours of traffic in seconds

*Your load tests now match production behavior — not synthetic guesses.*

# How It Works

## Simple 3-Step API

### Usage

---

```
// 1. Fetch timing from a real HCS topic
data, _ := hcsreplay.FetchTiming(ctx, "0.0.120438", hcsreplay.Mainnet, 1000)

// 2. Create a replay at 10x speed
replay := hcsreplay.NewReplay(data, hcsreplay.ModeSample, 10.0)

// 3. Use delays in your load test
for i := 0; i < 100; i++ {
    time.Sleep(replay.NextDelay())
    // Submit your HCS message here
}
```

### Technical Highlights

---

| Feature           | Detail                                         |
|-------------------|------------------------------------------------|
| Zero dependencies | Pure Go stdlib — no external packages required |
| SDK agnostic      | Works with any Hedera SDK; handles timing only |
| Network support   | Mainnet, Testnet, Previewnet                   |
| Test coverage     | 91.7% — production-ready quality               |
| License           | MIT — no restrictions on commercial use        |

# Innovation & Feasibility

## Judging Criteria

| Criteria    | How We Address It                                                                                                                                                                                                                                                                                 |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Innovation  | First library to capture and replay real HCS timing patterns for load testing — nothing like it exists in the Hiero ecosystem.                                                                                                                                                                    |
| Feasibility | Production-ready: 91.7% test coverage, CI/CD pipeline, comprehensive error handling, MIT-licensed with DCO compliance.<br>Synthetic timing generation uses log-normal distribution modeling (matching real network traffic patterns), with full statistical validation (min/max/avg/p50/p90/p99). |

## Why This Is Novel

- Hedera's Mirror Node exposes historical message timing. No existing tool leverages this for load testing.
- Extracted from production grpc-rest-benchmark tool — validated against real mainnet data
- Statistical replay modes capture realistic traffic variance that synthetic tools miss entirely

*Zero external dependencies means any Go project can go get and use it in minutes.*

# Execution & Integration

## Judging Criteria

| Criteria    | How We Address It                                                                                                 |
|-------------|-------------------------------------------------------------------------------------------------------------------|
| Execution   | Clean architecture, documented public API (godoc), DCO-compliant signed commits, GitHub Actions CI, golangci-lint |
| Integration | Uses Hedera Mirror Node REST API directly; designed to complement any Hedera SDK without replacing it             |

## Live Demo

---

- Demo repo: [github.com/kaldun-tech/hiero-hcs-replay-demo](https://github.com/kaldun-tech/hiero-hcs-replay-demo)
- Creates a test topic on Hedera testnet
- Fetches timing from a real mainnet topic (0.0.120438)
- Replays timing with actual HCS message submissions

# Validation & Success

## Judging Criteria

| Criteria   | How We Address It                                                                                                                                            |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Validation | Extracted from <a href="#">grpc-rest-benchmark</a> . This is a production benchmarking tool built against real Hedera mainnet workloads, not a toy prototype |
| Success    | Enables better QA → more robust dApps → higher user confidence in Hedera ecosystem                                                                           |

## Projected Network Impact

| Metric                       | Target                                                  |
|------------------------------|---------------------------------------------------------|
| Test accounts per project    | 10–100                                                  |
| TPS generation (test cycles) | 1K–10K per run                                          |
| Ecosystem benefit            | Better QA → fewer production failures → higher adoption |

# Roadmap

*Building on the foundation*

| Version                    | Features                                                                                            |
|----------------------------|-----------------------------------------------------------------------------------------------------|
| v1.0 — Current (Hackathon) | Core fetch/replay, Sequential + Sample modes, synthetic data generation, 91.7% test coverage, CI/CD |
| v1.1 — Post-Hackathon      | CLI wrapper for command-line usage, Go test framework integration, HTS token timing support         |
| v2.0 — Future              | WebSocket live traffic capture, Prometheus/Grafana metrics export, multi-topic correlation          |

## Go-To-Market

---

- Submit to awesome-hedera curated community list
- Publish technical blog post on Hedera Developer Portal
- Propose integration with Hedera Agent Kit
- Present at Hedera developer meetups and future hackathons
- Upstream to official Hiero tooling repository



# Demo

*See it in action*

 [Watch the Demo](#)

## What you'll see

---

- Fetching timing from a real mainnet HCS topic (0.0.120438)
- Statistical analysis output: message count, rate, P50/P90/P99
- Creating a replay at configurable speed and generating delays
- Live HCS message submission on testnet via demo repo

## Try it yourself

---

```
go get github.com/kaldun-tech/hiero-hcs-replay
```

# Links & Contact

| Resource           | URL                                                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Library Repository | <a href="https://github.com/kaldun-tech/hiero-hcs-replay">github.com/kaldun-tech/hiero-hcs-replay</a>                       |
| Demo Repository    | <a href="https://github.com/kaldun-tech/hiero-hcs-replay-demo">github.com/kaldun-tech/hiero-hcs-replay-demo</a>             |
| Documentation      | <a href="https://pkg.go.dev/github.com/kaldun-tech/hiero-hcs-replay">pkg.go.dev/github.com/kaldun-tech/hiero-hcs-replay</a> |
| On-Chain Proof     | <a href="https://hashscan.io/testnet/topic/0.0.8098128">hashscan.io/testnet/topic/0.0.8098128</a>                           |
| Website            | <a href="https://taras-smereka.dev">taras-smereka.dev</a>                                                                   |
| LinkedIn           | <a href="https://linkedin.com/in/taras-smereka">linkedin.com/in/taras-smereka</a>                                           |
| Discord            | kaldun_tech                                                                                                                 |
| Substack           | <a href="https://taraskaldun.substack.com">taraskaldun.substack.com</a>                                                     |

**Zero dependencies. MIT licensed. Ready for production.**

# Submission Details

## 100-Word Project Description & Tech Stack

### Project Description (98 words)

---

*hier-hcs-replay is a Go library that captures real message timing patterns from Hedera Consensus Service (HCS) topics and replays them for realistic load testing. Unlike synthetic load generators that produce uniform traffic, this library fetches actual timing data from production HCS topics via the Mirror Node REST API, providing statistical analysis (P50/P90/P99) and configurable replay speeds. Zero external dependencies, 91.7% test coverage, and MIT licensed. The library complements any Hedera SDK. It handles timing and pacing while developers use their preferred tools for message submission. Built for the Hiero ecosystem to help developers ship more robust Hedera applications.*

### Tech Stack

---

- Go 1.21+ (programming language)
- Go Standard Library (net/http, encoding/json, math, sync)
- Hedera Mirror Node REST API (data source)
- GitHub Actions (CI/CD)
- golangci-lint (code quality)